Designing & Debugging MLPs: From Architecture to Training - 2

Saturday, 17 October 2026 6:03 PM

Recap:

Orthogonalization: Tuning Hyperparameters of a neural network.

Random Search

Grid Search.

HW:

1) Gradient checking.

2) Micro & Macro F1

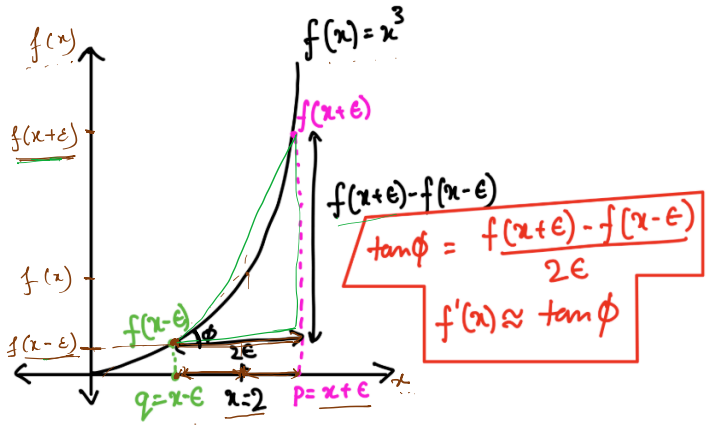
- Gradient Descent from Scratch. (there are chances of making mistake)
- Use auto-grad in PyTorch. → chances of making a mistake in implementation is almost 0.

Q: How do we identify if our backward propogation and gradient implementation is correct?

→ We use gradient checking.

Gradient checking → It is a technique used to verify whether the gradients calculated by backpropogation are correct by comparing them with numerical approximation of the gradients.

$$f(x) = x^3$$



$x = 2$

$q = x - \varepsilon$ ⟩ small value.

$p = x + \varepsilon$

$\varepsilon = 0.01$

for the triangle,

base = 2ε

height = $f(x+\varepsilon) - f(x-\varepsilon)$.

height = $f(x+\varepsilon) - f(x-\varepsilon)$

$$\tan\phi = \frac{f(x+\varepsilon) - f(x-\varepsilon)}{2\varepsilon} \rightarrow \frac{\text{height}}{\text{base}} \quad (\text{approx}).$$

↓ small delta.

if $\varepsilon = 0.01$

then $\tan\phi$,

$$\frac{(2.01)^3 - (1.99)^3}{2 \times 0.01} = 12.0001$$

actual gradient, $f'(x) = 3x^2 = 3 \cdot (2)^2 = 12.$

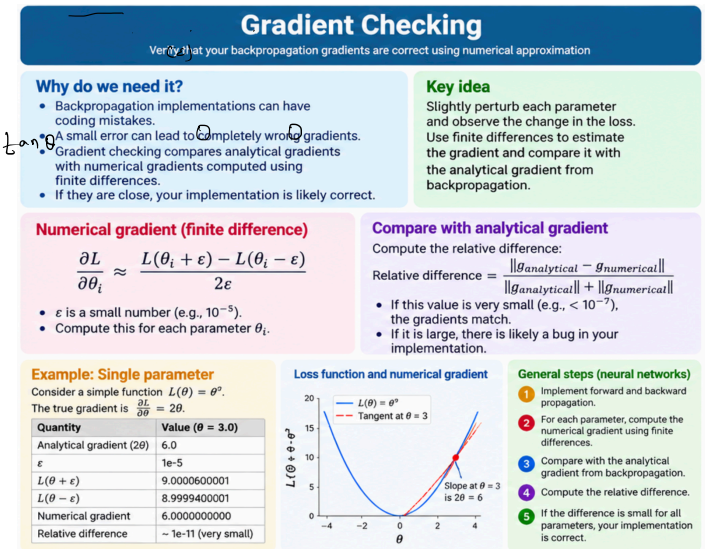
$$z = w^T x + b.$$

$\tan\theta$. $\frac{f(x+\varepsilon) - f(x-\varepsilon)}{2\varepsilon} \approx f'x$

$\frac{\partial L}{\partial w}$ = chain rule.

$12.0001 \simeq 12$ → This is called as gradient checking.

if the difference is less than 0.01 then the gradient calculation is correct.



* Steps in Gradient checking

1) Calculate the analytical gradient
   Use backprop and chain rule.

2) Calculate the numerical gradient
   Change a parameter slightly and observe the change in loss.

3) Compare the two gradients

Case 1

Small difference

We are good.

Case 2

Large difference

Investigate the implementation.

Quiz 1:

Inspect the backward code of that layer's weights

* Evaluation Metrics.

	Precision	Recall
Model 1 :	0.68	0.35
Model 2 :	0.64	0.40

→ Which model is good?

Combined eval → F1 score

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Model 1 = 0.462

Model 2 = 0.492.

→ Model 2 is better.

Micro F1 : Aggregate the TP, FP, FN across classes first.

Then calculate Precision, Recall, F1

Macro F1 : Calculate the F1 score independently for each class, then take a mean of the scores.

Micro vs Macro F1 Score

Two different ways to average class-wise F1 scores in multi-class classification

F1 score for a single class

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

where:

- Precision = $\frac{TP}{TP + FP}$
- Recall = $\frac{TP}{TP + FN}$

Key differences

- **Micro F1:** Aggregates the contributions of all classes before computing precision and recall. Gives more weight to frequent classes.
- **Macro F1:** Computes the F1 score for each class independently and takes the unweighted average. Gives equal weight to all classes, including minority classes.

Example: 3-class classification

Confusion Matrix

Actual class \ Predicted class	A	B	C
A	50	5	0
B	10	20	0
C	0	5	10

Per-class precision, recall and F1

Class	Precision	Recall	F1 Score
A	0.83	0.91	0.87
B	0.67	0.67	0.67
C	1.00	0.67	0.80

Macro F1 Score (unweighted average)

Compute F1 for each class and take the average.

$$\text{Macro F1} = \frac{0.87 + 0.67 + 0.80}{3} = 0.78$$

Each class contributes equally, regardless of how many samples it has.

Micro F1 Score (aggregate counts)

Sum up TP, FP and FN across all classes, then compute precision, recall and F1.

- Total TP = 50 + 20 + 10 = 80
- Total FP = 5 + 10 + 5 = 20
- Total FN = 5 + 10 + 5 = 20

$$\text{Precision} = \frac{80}{80 + 20} = 0.80$$

$$\text{Recall} = \frac{80}{80 + 20} = 0.80$$

$$\text{Micro F1} = \frac{2 \times 0.80 \times 0.80}{0.80 + 0.80} = 0.80$$

Frequent classes have more influence on the final score.

Imbalanced Data → Use Macro F1

A single-label classifier with four classes always predicts the majority class, which covers 90% of the data. Which metric exposes this problem?

} Imbalanced Dataset.
↓
Use Macro F1.

Models	Accuracy	Latency
A	.80	100 ms
B	.85	150 ms
C	.95	250 ms

→ Model C is best.

If in Production we can have up to 200 ms latency.

Now? → Use model B.

ETA prediction model → A hard cutoff at 100 ms

↑ 100 requests.
80 is < 100 ms
20 is > 100 ms
error rate → 20% → 95% of requests should be served within 100 ms.

* Human Level Performance.

Model → XRay Diagnosis.

A doctor is 80% accurate.

We want model performance > human level performance.

Suppose a model has : Training Error → 7%.
Val Error → 9%.

Human performance error → 1%.

the training − human error = (7 − 1) = 6%.

Large delta in the training performance. → Model Underfits

Scenario 2 :

Model training error → 2%
Val error → 8%
Human Error → 1%

→ Training is fine.
→ High Train & Val gap.

Reduce Variance → Use Regularization
Use more data.

For a task, the best human expert team makes 2% error. A model has 3% training error and 8% validation error. What should you work on first?

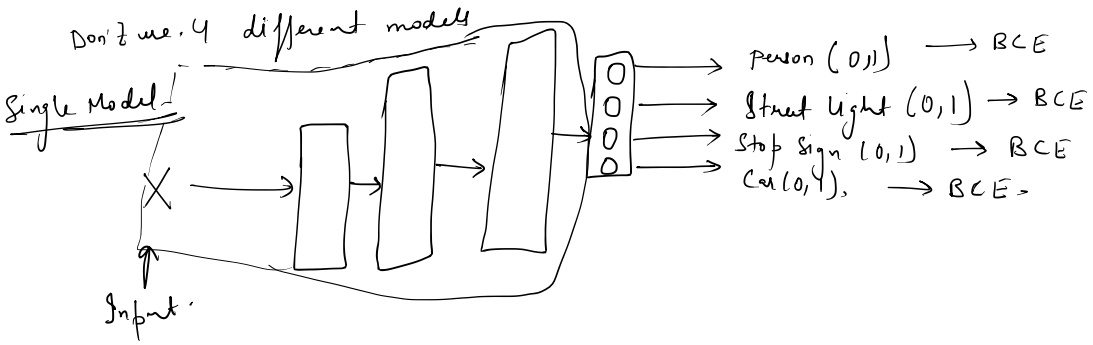
} Overfitting
↓
Reduce the variance.

* Multi Task learning.

Image (0,1).

→ car (0,1).

4 Classes → person, Street light, stop sign, car.



Loss → Binary Cross Entropy.

Advantages :

- Reduced complexity ✓